Sanskrit RAG Project

1. Transformer

A deep learning architecture used in models like GPT, BERT, and TinyLlama.

It understands sentence meaning, word relationships, and context.

Model used: TinyLlama/TinyLlama-1.1B-Chat-v1.0

2. RAG (Retrieval-Augmented Generation)

Combines retrieval and text generation.

Flow: User Question → Retriever → LLM reads context → Final Answer.

Purpose: Reduce hallucination, improve factual accuracy.

3. Document Loader

Loads files into LangChain. Uses Docx2txtLoader to read the .docx Sanskrit file.

Example: loader = Docx2txtLoader('Rag-docs.docx')

4. Chunking

Splits large documents into smaller pieces so LLMs can process them.

Uses RecursiveCharacterTextSplitter with chunk_size=500, chunk_overlap=50.

5. Embedding Model

Converts text into numerical vectors for semantic similarity search.

Model used: intfloat/multilingual-e5-base

Understands Sanskrit, Hindi, English, and other languages.

6. Vector Database

Stores embeddings for fast semantic search.

This project uses Chroma (ChromaDB).

Flow: Text Chunk → Embedding Vector → Stored in ChromaDB.

7. Retriever

Finds relevant chunks from the vector DB for a given query.

Setting: search_kwargs={'k': 3} — retrieves top 3 relevant chunks.

8. Similarity Search

Compares query embedding with document embeddings to find the closest match.

Example: 'dog' and 'puppy' can match semantically.

9. HuggingFace Pipeline

Simplifies loading and running a model.

Used as: pipeline(task='text-generation')

10. TinyLlama

A lightweight LLM that runs on CPU.

Advantages: Small size, no GPU needed, fast inference.

Limitations: Less accurate than GPT-4, more hallucinations possible.

11. Prompt Engineering

The prompt controls model behavior.

Key instruction: 'Answer ONLY from the given context.'

Prevents hallucination and restricts outside knowledge.

12. Hallucination

When an LLM generates false or imaginary information.

Prevented here using RAG, strict prompts, and retrieval-based answering.

13. Context Window

The retrieved chunks passed to the LLM before generating an answer.

The model only reads this information — nothing else.

14. Semantic Search

Matches meaning rather than exact keywords.

Traditional search: keyword matching. Semantic search: meaning matching.

15. Accuracy Evaluation

Formula: Accuracy = (Correct Answers / Total Questions) x 100 Example: 45 correct out of 50 = 90% accuracy.

16. Excel-Based Evaluation

A spreadsheet tracking: Question, Expected Answer, Model Answer, Status.

Used for performance tracking and manual verification.

17. LangChain

Framework for building LLM applications.

Used for document loading, retrieval pipelines, prompt chaining, and RAG workflows.

18. ChromaDB Persistence

Saves the vector database to disk so embeddings are not rebuilt every run.

Setting: persist_directory='./sanskrit_db'

19. StrOutputParser

Cleans raw LLM output into plain readable text.

20. Runnable Chain

LangChain pipeline: Question → Retriever → Prompt → LLM → Final Answer.

Invoked using: rag_chain.invoke()

21. CPU-Based Inference

Model runs on CPU (device=-1), no GPU required.

Advantage: Works on normal laptops. Disadvantage: Slower inference.

22. Multilingual Embedding

The embedding model supports Sanskrit, Hindi, English, and more.

Useful because user queries can come in different languages.

23. Retrieval Quality

Measures how relevant the retrieved chunks are.

Improved by better embeddings, proper chunking, and good retriever settings.

24. Top-k Retrieval

k controls how many chunks are retrieved. This project uses k=3.

Higher k = more context but more noise. Lower k = faster but may miss info.

25. End-to-End RAG Workflow

Document → Chunking → Embeddings → Vector DB → Retriever → Prompt → LLM → Answer.

Each step feeds into the next to form the complete pipeline.